

```

curr_balance[account] = curr_balance[account] - money;
pthread_mutex_unlock(&account_lock /* for that account */);
}

void Deposit (int account, int money)
{
    pthread_mutex_lock(&account_lock /* for that account */);
    //get the curr_balance for this account and increment it
    curr_balance[account] = curr_balance[account] + money;
    pthread_mutex_unlock(&account_lock /* for that account */);
}

```

The above code contains two functions: *Withdraw*, which debits a bank account with a specified amount of money by decrementing the shared variable *curr_balance*; and the function, *Deposit*, which increments *curr_balance* by the amount being deposited. As we can see, both the accesses to *curr_balance* are properly protected by the same lock *account_lock* and everything seems absolutely correct. So what can be the problem?

A new developer maintaining the code decides to add a function called *transfer*, which allows a customer to transfer money from account A to account B. He implements the function, *transfer*, as follows:

```

void Transfer (int from_account, int to_account, int money)
{
    Withdraw(from_account, money);
    Deposit(to_account, money);
}

```

What is wrong with the above piece of code? Well, since we have a multi-threaded program, let us assume that we have a thread that queries and adds up the total balance in all accounts. Let us further assume that this thread queries the *curr_balance* in each account to calculate the total balance. If this thread is scheduled and queries the *curr_balance* in *from_account* and *to_account* in the time period between the call to *Withdraw* and the call to *Deposit* in the function *transfer*, it will find that *curr_balance* in the *from_account* has been debited while the *curr_balance* in the *to_account* has not been incremented yet. There is this time window where the money is missing completely from the books of the bank and the balance reporting thread will report incorrect results.

So where did we go wrong? We did acquire the locks correctly in both the *Withdraw* and *Deposit* functions. However, we missed the point that the function *Transfer* should be an atomic operation, and that both debiting the *from_account* and crediting the *to_account* should be done atomically. Hence the correct code for *Transfer* should look as follows:

```

void Transfer (int from_account, int to_account, int money)
{

```

```

    pthread_mutex_lock(&account_lock /* for 'from_account' */);
    pthread_mutex_lock(&account_lock /* for 'to_account' */);
    curr_balance[from_account] = curr_balance[from_account] - money;
    curr_balance[to_account] = curr_balance[to_account] + money;
    pthread_mutex_unlock(&account_lock /* for 'from_account' */);
    pthread_mutex_unlock(&account_lock /* for 'to_account' */);
}

```

Transferring money from one account to another needs to be performed as a single atomic transaction. Hence the locks required for both the accounts need to be held while we debit one account and credit the other. Else the system state as seen by an external entity can be inconsistent, as we realised in the example above. We will continue our discussion on atomicity, race conditions and deadlock in next month's column.

This month's takeaway problem

This month's takeaway problem is again to do with multi-threaded code and is quite simple. Can you detect what may be the issue in the following code snippet:

```

void BookTicket(int row, int column)
{
    pthread_mutex_lock(&row_lock);
    pthread_mutex_lock(&column_lock);
    ticket[row][column].status = 'booked';
    pthread_mutex_unlock(&column_lock);
    pthread_mutex_unlock(&row_lock);
}

void CancelTicket(int row, int column)
{
    pthread_mutex_lock(&column_lock);
    pthread_mutex_lock(&row_lock);
    ticket[row][column].status = 'cancelled';
    pthread_mutex_unlock(&row_lock);
    pthread_mutex_unlock(&column_lock);
}

```

Note that two threads can concurrently issue booking and cancellation requests to the same ticket (same row and same column) and the system has to work correctly.

If you have any favourite programming puzzles that you would like to discuss on this forum, please send them to me. Also, do send your solutions and feedback to sandyasm_AT_yahoo_DOT_com. Till we meet again next month, happy programming! 

About the author:

Sandya Mannarswamy. The author is a specialist in compiler optimisation and works at Hewlett-Packard India. She has a number of publications and patents to her credit, and her areas of interest include virtualisation technologies and software development tools.